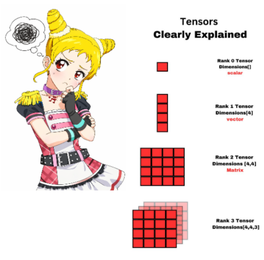
Day 2
100 Day Deep Learning THEORY

Tensor - A,B,C for Deep Learning



Container for Numbers

Generalization of Scalar, Vector, Matrix

Multi-dim

tips : understand tensor as how much
axis needed to represent that number

8 - 0 axis (point)
[1,8,45,] - 1 axis (x axis example)

[[1,2],[8,0]] - 2 axis (x-y axis example)

 # Import Pytorch
import torch

1) Tensor

7

1. **Dimension (Rank):** 0
2. **Shape:** ()
3. **Meaning:** Single number
4. **Real use:** Loss value in training

[10 , 20 , 30]

1. **Dimension (Rank):** 1
2. **Shape:** (3,)
3. **Meaning:** List of numbers
4. **Real use:** Audio signal, feature list

[1 2 3]
[4 5 6]

1. **Dimension (Rank):** 2
2. **Shape:** (2, 3)
3. **Meaning:** Table (rows × columns)
4. **Real use:** Dataset

Image =
[
[[255, 0, 0], [0, 255, 0]],
[[0, 0, 255], [255, 255, 0]]
]

1. **Dimension (Rank):** 3
2. **Shape:** (Height, Width, Channels) → (2, 2, 3)
3. **Meaning:** Color image (RGB)
4. **Real use:** Computer vision

 # for dim :
Count no of elements in box

(1,4,7,9) = 4 dim
(1, 6) = 2 dim
(9,6,9,0,0) = 5 dim

 # naming :
(9, 3, 64, 64) = (Batch, Channel, Seq, Element)
say there are 9 batches, each batch have 3 channels
, each channel have 64 seq, and each seq have 64
elements

2. Create Tensor

Tensor from Input

```
t0 = torch.tensor(5)
```

```
t1 = torch.tensor([10, 20, 30])
```

```
t2 = torch.tensor([
    [1, 2, 3],
    [4, 5, 6]
])
```

```
t3 = torch.tensor([
    [[255, 0, 0], [0, 255, 0]],
    [[0, 0, 255], [255, 255, 0]]
])
```

From Numpy

```
import torch
a1 = np.array([10, 20, 30])
torch_tensor = torch.from_numpy(a1)
print(torch_tensor)
```

torch.from_numpy( )

Exact like Tesors

z = torch.zeros(2, 3) → [[0. 0. 0.]
[0. 0. 0.]]

o = torch.ones(2, 3)

e = torch.empty(2, 3)

f = torch.full((2, 3), 7)

Arrange Number Tesor

a = torch.arange(0, 10, 2) → [0 2 4 6 8]

l = torch.linspace(0, 10, 5) → [0. 2.5 5. 7.5 10.]

lg = torch.logspace(0, 3, 4) → [1. 10. 100. 1000.]

$10^0 \sim 10^3$

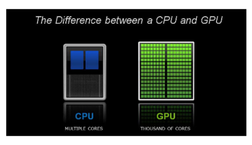
I = torch.eye(3) → [[1. 0. 0.]
[0. 1. 0.]
[0. 0. 1.]]

d = torch.diag(torch.tensor([2, 4, 6])) → [[2 0 0]
[0 4 0]
[0 0 6]]

r = torch.rand(2, 3) → [[0.23 0.87 0.45]
[0.66 0.12 0.91]]

3.Hardware Acceleration in Tensor

Unlike Numpy
Tensor could live in **CUDA** and could be **Trained in Parallel**



CUDA (Compute Unified Device Architecture) is NVIDIA's parallel computing platform and programming model that lets developers use the **massive parallel processing power of their GPUs**

AI RUNNING ON CPU

AI RUNNING ON GPU

How computer processors work

Multiply two matrices
of size **5000000 × 500000000**

CPU = 24 hours

GPU = 1 second

15 sec = 0.1 second
1000 years = 1 day

torch.cuda.is_available() → True ✓ / False

torch.cuda.get_device_name(0)
Check GPU name

Use GPU if available
Or use CPU only

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

 # Send a Tensor to CUDA

x = torch.tensor([1.0, 2.0, 3.0])
x = x.**to(device)**

x = torch.zeros((3, 3)**, device="cuda"**)

Q) Will it Work ? **(PYQ Sep 2025 Qz2)**

a = torch.tensor([1.0]).cuda()

b = torch.tensor([2.0])

c = a + b

✗ Run it

 # Back to CPU

x_cpu = x.**cpu()**

So Why Tensor ?

Tensor = Numpy + SuperPower + AutoGrad

3.Tensor Manipulation

X

[[1, 2],
[3, 4],
[5, 6]]

1. Reshaping Operations

x.**view**(3, 2) → [[1, 2],
[3, 4],
[5, 6]]

x.**reshape**(1, 6) → [[1, 2, 3, 4, 5, 6]]

[[1, 2, 3],
[4, 5, 6]]

x.**unsqueeze**(0) → [[[1, 2, 3],
[4, 5, 6]]] → y.**squeeze**() → [[1, 2],
[3, 4],
[5, 6]]

(2, 1) → UN → (1, 2, 1) → unsq → (1, 1, 2, 1)

2.Indexing & Slicing

[[1, 2, 3],
[4, 5, 6]]

x[0] [1, 2, 3]

x[:, 1] [2, 5]

x[0, 2] 3

3.Mathematics

[[1, 2, 3],
[4, 5, 6]]

x + 10 → [[11, 12, 13],
[14, 15, 16]]

x * 2 → [[2, 4, 6],
[8, 10, 12]]

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

a = torch.tensor([[1, 2],
[3, 4]])
b = torch.tensor([[5, 6],
[7, 8]])

torch.**matmul**(a, b) → [[19, 22],
[43, 50]]

4.Sorting

torch.**sort**(torch.tensor([3, 1, 2]))

torch.**topk**(torch.tensor([10, 50, 20]), k=2)

C. Transpose and Cloning

x.transpose $\begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix}$

A. Reduction Opearations

x.sum() # Sum all Values in Tensor

$\begin{bmatrix} 1 & 2 \\ 4 & 3 \end{bmatrix}$

B. Joining

torch.cat([a,c]) # Making full tensor of element

torch.stack([a,c]) # Stacking one tensor above tother

$X_1 = X$ #referencing Danger

$X_1 = X.\text{clone}$

5.Autogradient

It automatically computes **gradients,**of tensors **during backpropagation**.

x = torch.tensor(2.0, requires_grad=True)

y = x ** 2 + 3 * x + 1 $y = x^2 + 3x + 1$

y.backward()

$2x + 3 = \frac{dy}{dx}$

Since $x = 2$
$2(2) + 3 = 7$ ✓

x = torch.tensor(2.0, requires_grad=True)

y = x ** 2
y.backward()
print(x.grad) # 4

$x^2 = 2x = 4$

y = x ** 3
y.backward()
print(x.grad) # 4 + 12 = 16

$x^3 = 3x^2 = 3(2)^2 = 12$

 # Stop Gradient = with torch.no_grad():
do something.....

 # Remove Gradient = x.grad.zero_()

5.Numpy and Tensor Integration

np_array = tensor.**numpy()** → NumPy

tensor = torch.**from_numpy**(np_array)

✗ **If tensor is in GPU**
Run time Error

np_array = tensor.**detach().cpu().numpy()**

7.Reproducibility

import random

seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed(seed)

***Helpful duringf Research or Practical Submission**
where Random values need to be consistent

Thanks for Watching

See you in Practice SESSION for this Tensor
(its very important)